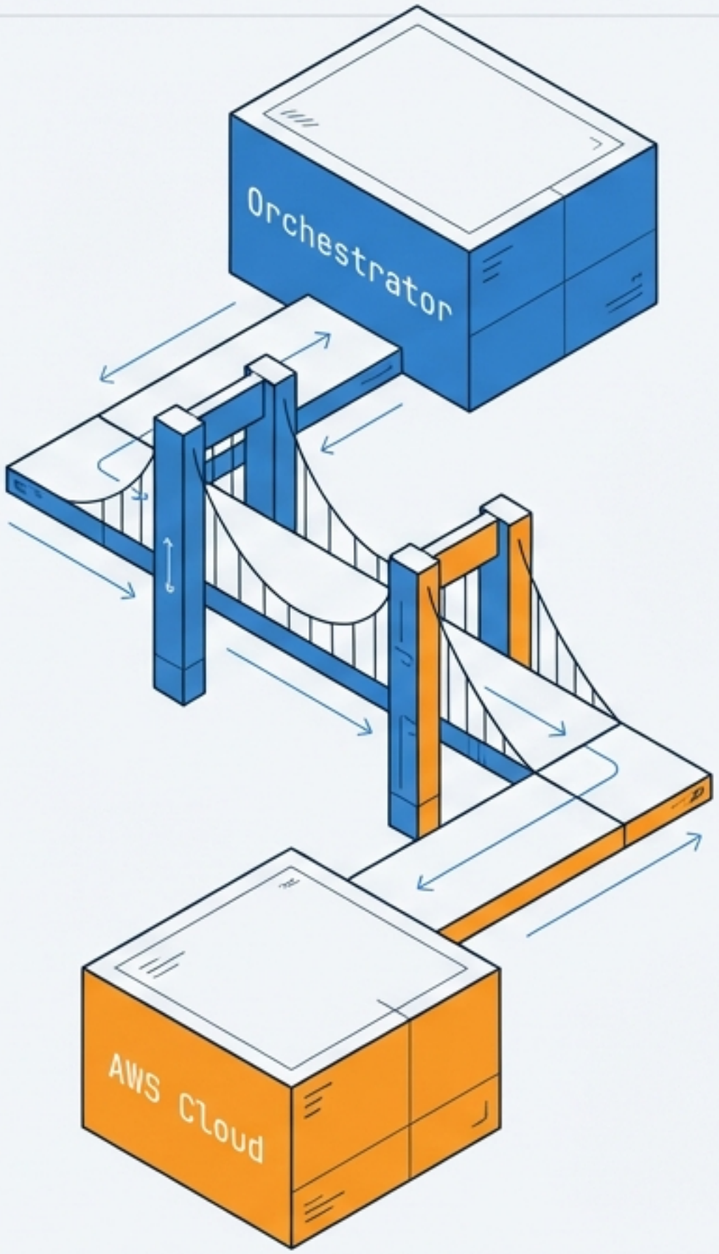
HTML UI Service for AWS Deployment

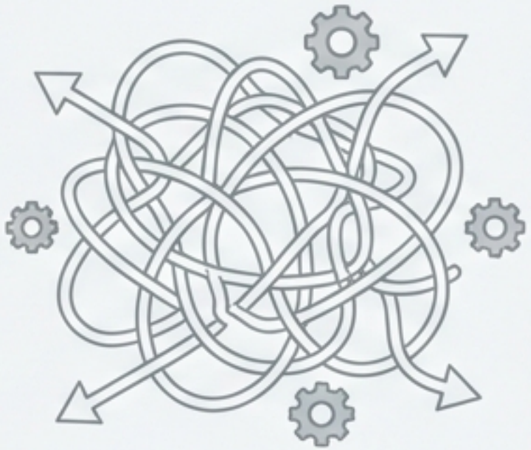
Bridging Build Orchestration and On-Demand Infrastructure

A technical specification for a web-based interface and RESTful API designed to automate the lifecycle of AWS environments. Leveraging OSBot-AWS to decouple high-level build logic from low-level infrastructure execution.



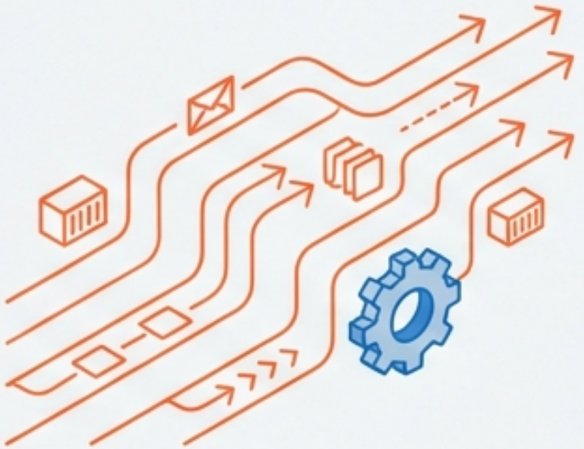
Enabling On-Demand Application Infrastructure

Current State: Rigid & Manual



- High manual intervention required for provisioning.
- Static environments leading to resource waste.
- Complex, brittle deployment scripts.

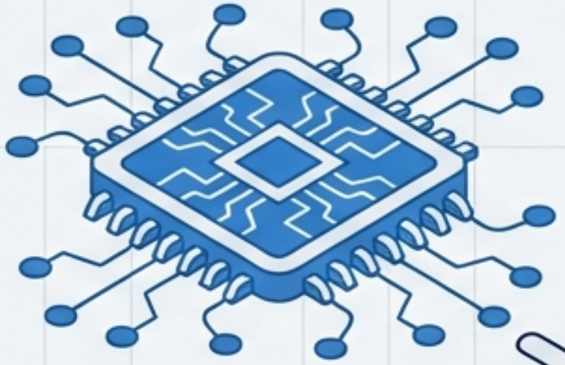
Future State: Dynamic & Automated



- Fully automated, code-driven lifecycle.
- Dynamic scaling: Spin up/tear down on demand.
- Separation of concerns: Logic vs. Execution.

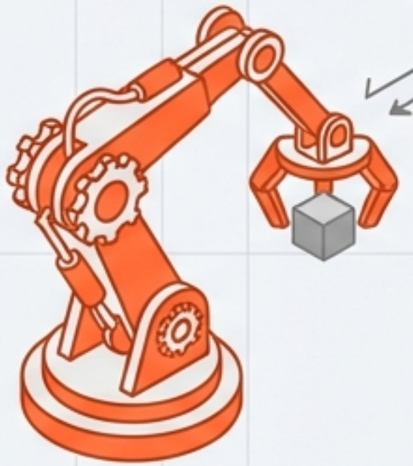
Primary Objective: Empower users and automated processes to provision entire application stacks in any region via simple API calls, integrating tightly with existing build orchestrators.

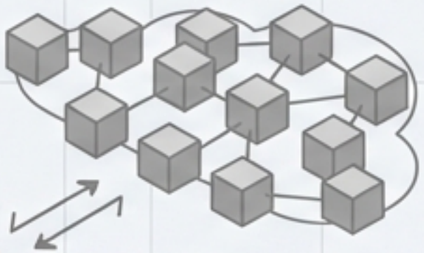
Architectural Philosophy: The Brain and The Hands



THE BRAIN
(Build Orchestrator)

Responsible for Logic.
Determines sequence,
timing, and "When" to act.
Holds workflow state.





AWS Resources

THE HANDS
(UI Service)

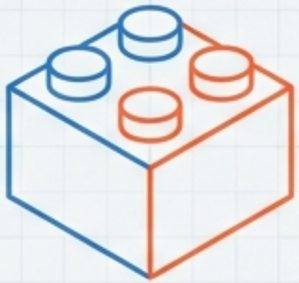
Responsible for Execution.
Translates requests into AWS primitives.
Handles "How" to act. Stateless.

Core System Design Principles

Statelessness

The service does not track long-term infrastructure state (like Terraform state files). Each request is independent, allowing for high scalability, parallel execution, and simpler recovery.

Primitive Operations



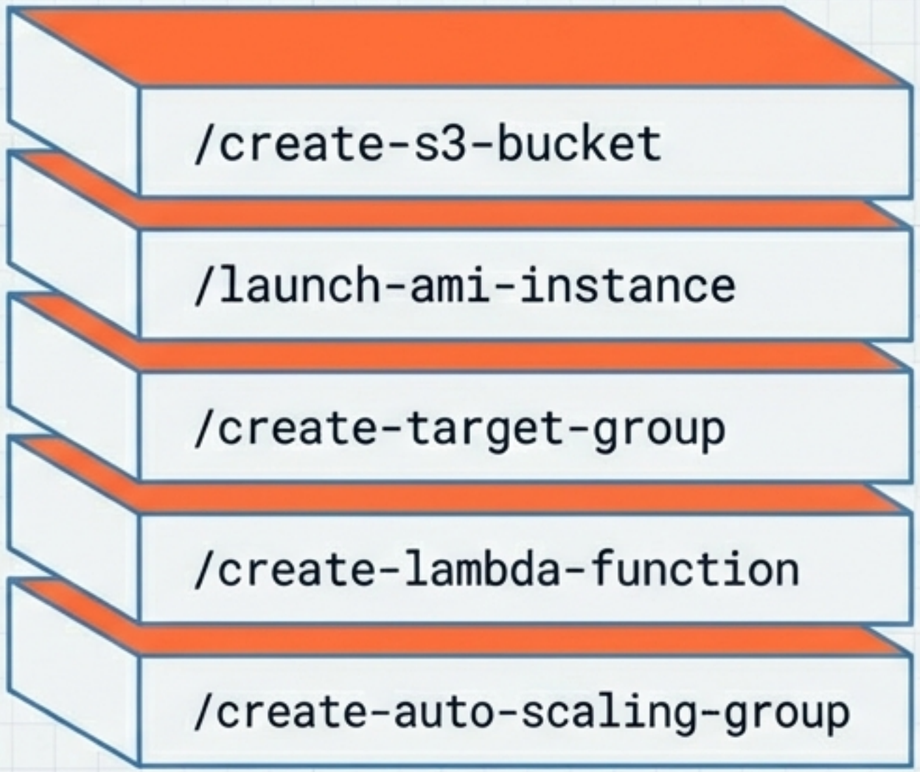
Exposes granular, atomic actions rather than opaque workflows. We build '/create-bucket', not '/deploy-app'. This prevents hardcoding business logic into the execution layer.

Secure Pass-Through

No persistent credential storage. The service acts as an ephemeral gateway, decrypting keys in memory only for the duration of the specific request.

The API Layer: AWS Primitives

Modular ‘Lego Blocks’ for Orchestration

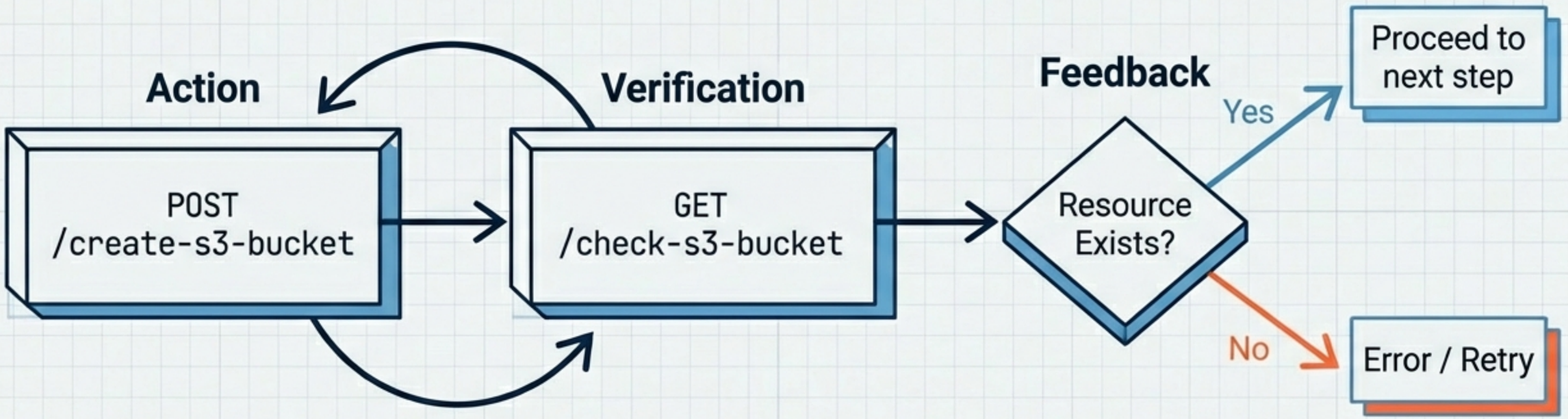


RESTful Design:

Every AWS capability is exposed as a discrete call. The Orchestrator composes these blocks into any sequence required, ensuring flexibility without modifying the UI Service code.

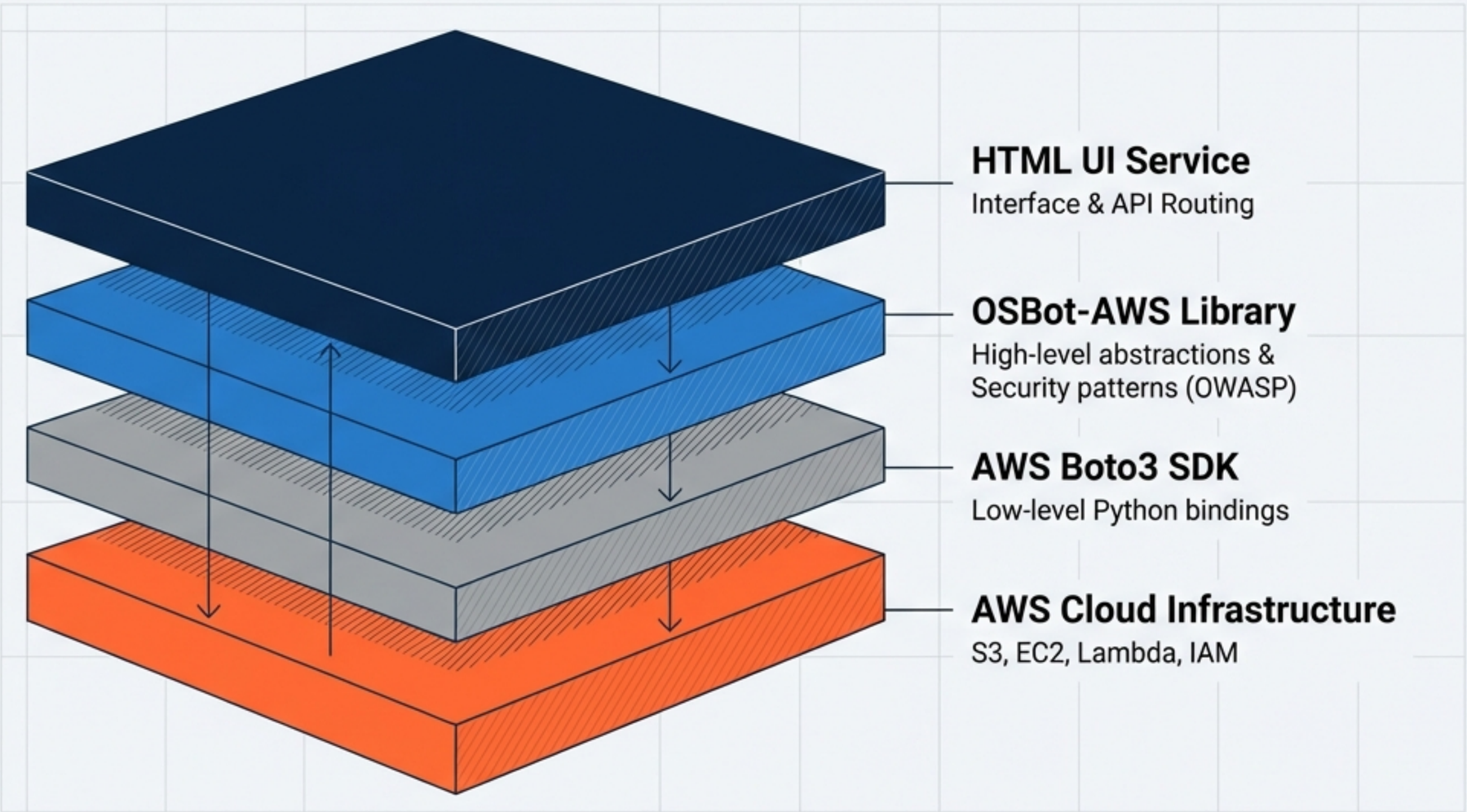
Also includes corresponding /check endpoints for verification.

The Verification Loop: Trust but Verify



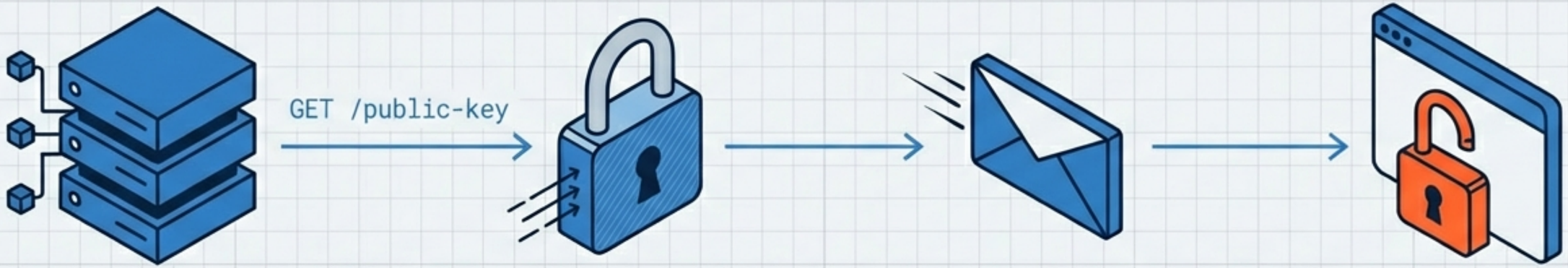
Reliability Mechanism: For every create action, there is a corresponding check action. This allows the orchestrator to confirm successful resource creaton before proceeding, enabling robust error handling and conditional logic.

Powered by OSBot-AWS



The service leverages the OSBot-AWS Python library to abstract low-level Boto3 complexities. This accelerates development and ensures consistent, safe interaction with AWS resources.

Security Architecture: Credential Pass-Through



1. Request Public Key

2. Encrypt AWS Keys

Orchestrator encrypts Access Key ID & Secret Key using the Service's Public Key.

3. Transmit Securely

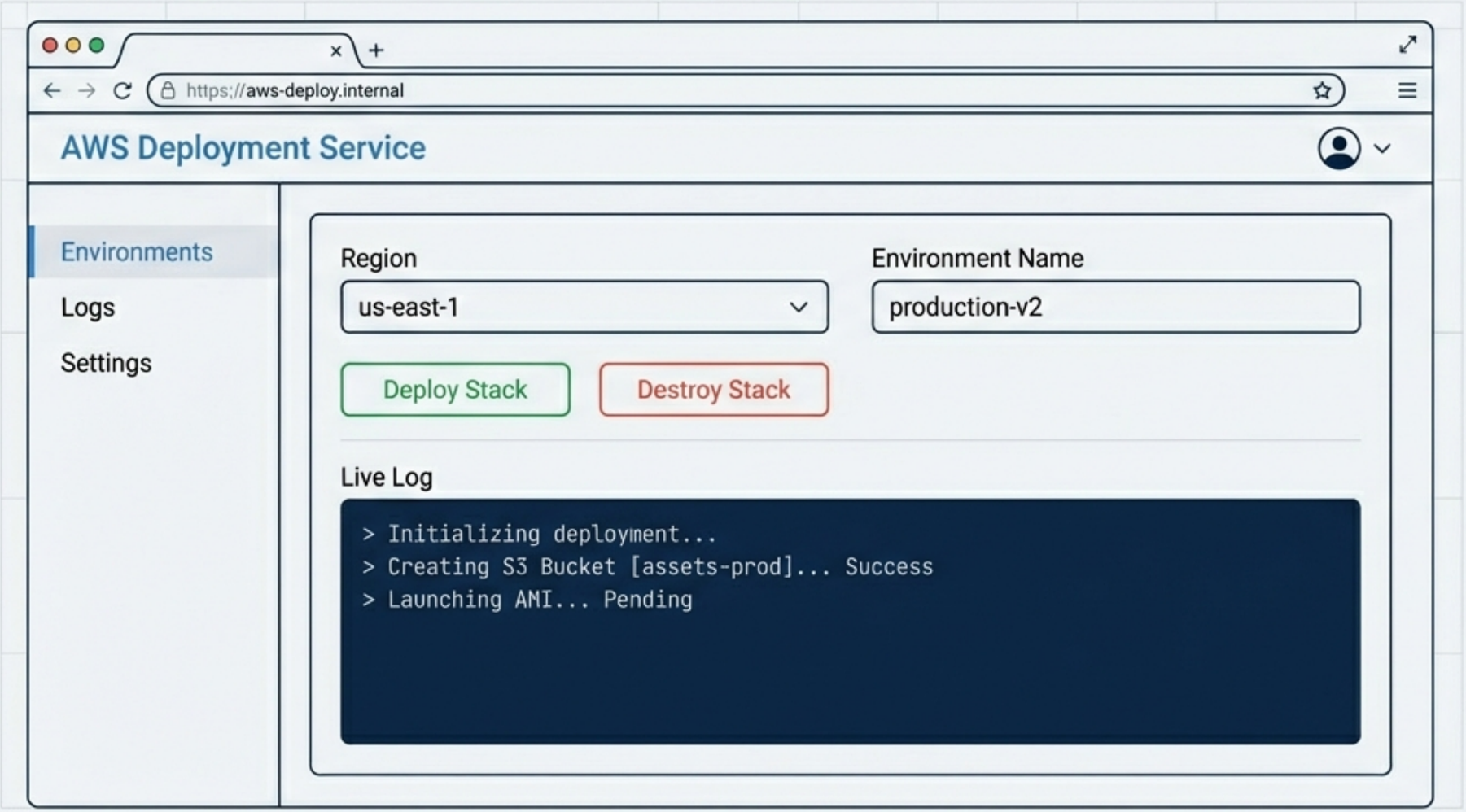
Encrypted payload sent to UI Service endpoints.

4. Decrypt & Execute

UI Service decrypts using Private Key. Credentials exist only in memory for the session.

Zero-Knowledge Storage: The UI Service never writes credentials to disk. The Orchestrator never exposes raw secrets.

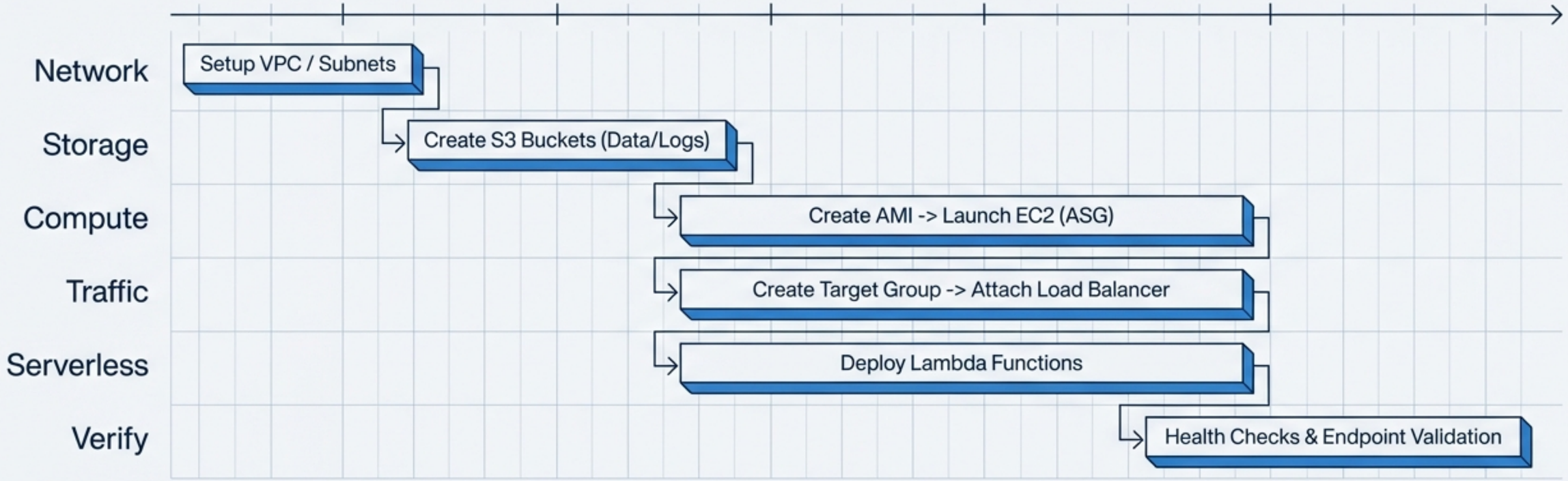
The Human Interface: Web Dashboard



While designed for automation, the HTML UI empowers developers to:

1. Manually trigger deployments.
2. Debug specific steps without AWS Console access.
3. View real-time execution logs.

Integration Scenario: A Full Stack Deployment



The Orchestrator strings together these primitives to build a full stack, verifying health at every stage before routing traffic.

Adaptive Deployment Modes

TOPOLOGY CREATION

Fresh Deployment

Build from scratch

Creates full infrastructure topology (VPC, Security Groups, Resources).

CONFIGURATION UPDATES

Update Mode

Modify existing

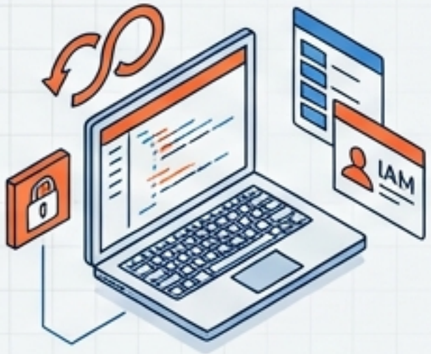
Skips creation of existing resources; updates configurations or code on Lambdas/EC2.

Secure Mode

Production Standard

Uses the public-key encryption flow for credential passing. No local secrets.

ENCRYPTION FLOW



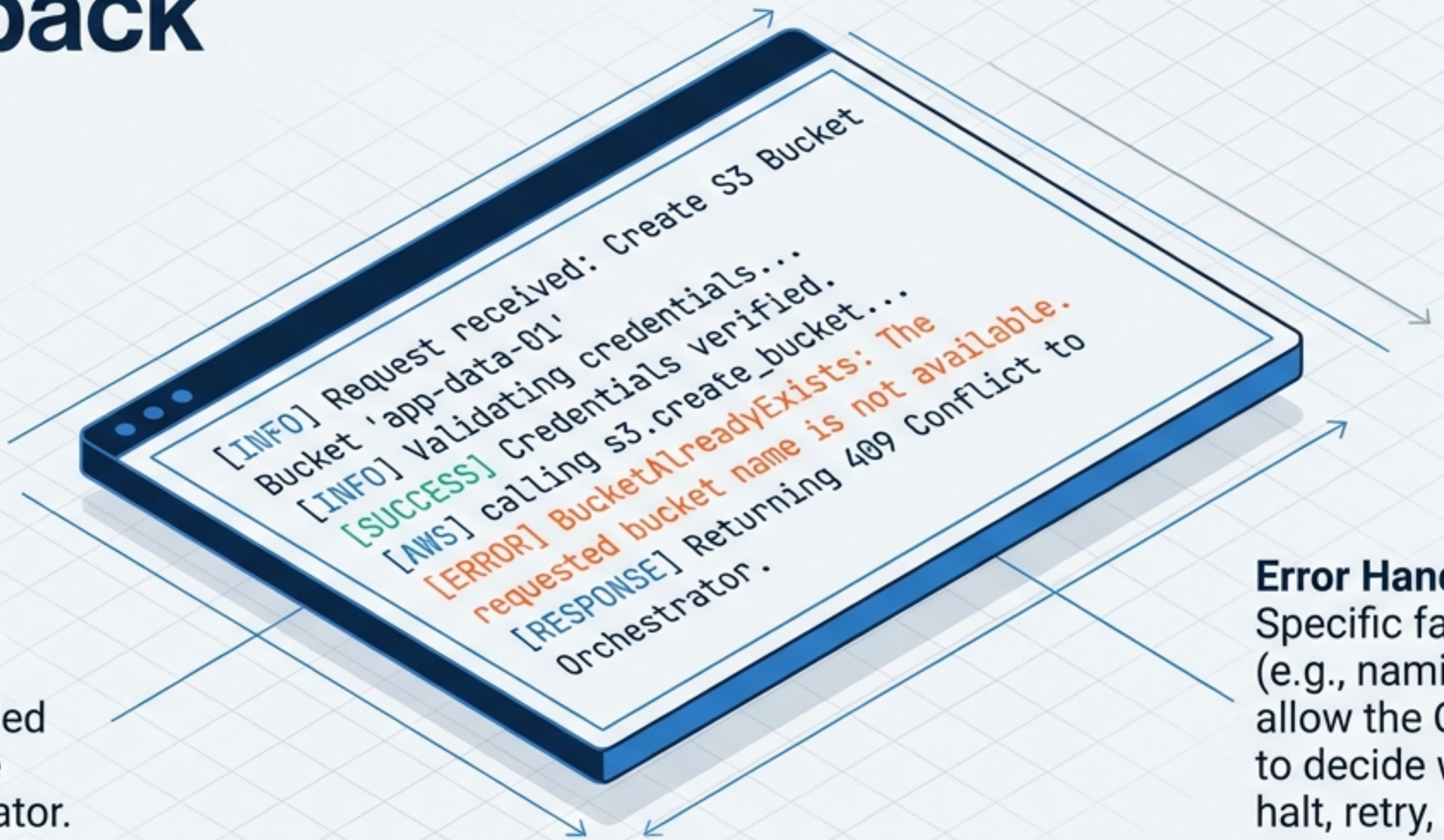
Dev / Test Mode

Rapid Iteration

Uses pre-configured environment variables or IAM roles to bypass encryption for local testing.

LOCAL BYPASS

Logging and Operational Feedback



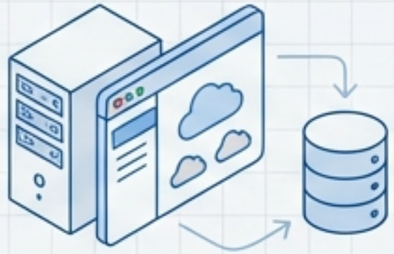
Total Visibility:
Every action produces detailed logs accessible to the Orchestrator.

Error Handling:
Specific failure states (e.g., naming conflicts) allow the Orchestrator to decide whether to halt, retry, or modify parameters.

Project Deliverables

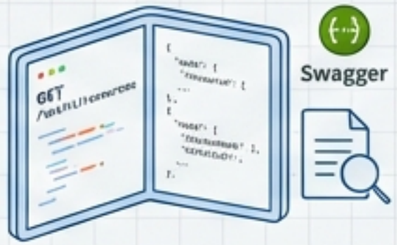
Functional Web Application

The running HTML UI service exposing AWS primitives via UI and API.



API Documentation (The Contract)

Swagger/OpenAPI specs detailing endpoints, JSON payloads, and response codes.

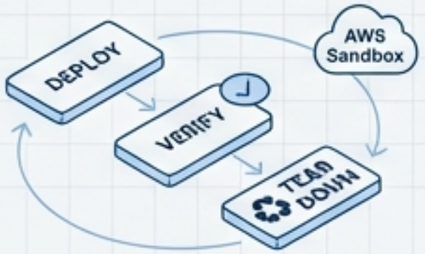


Security Audit Report

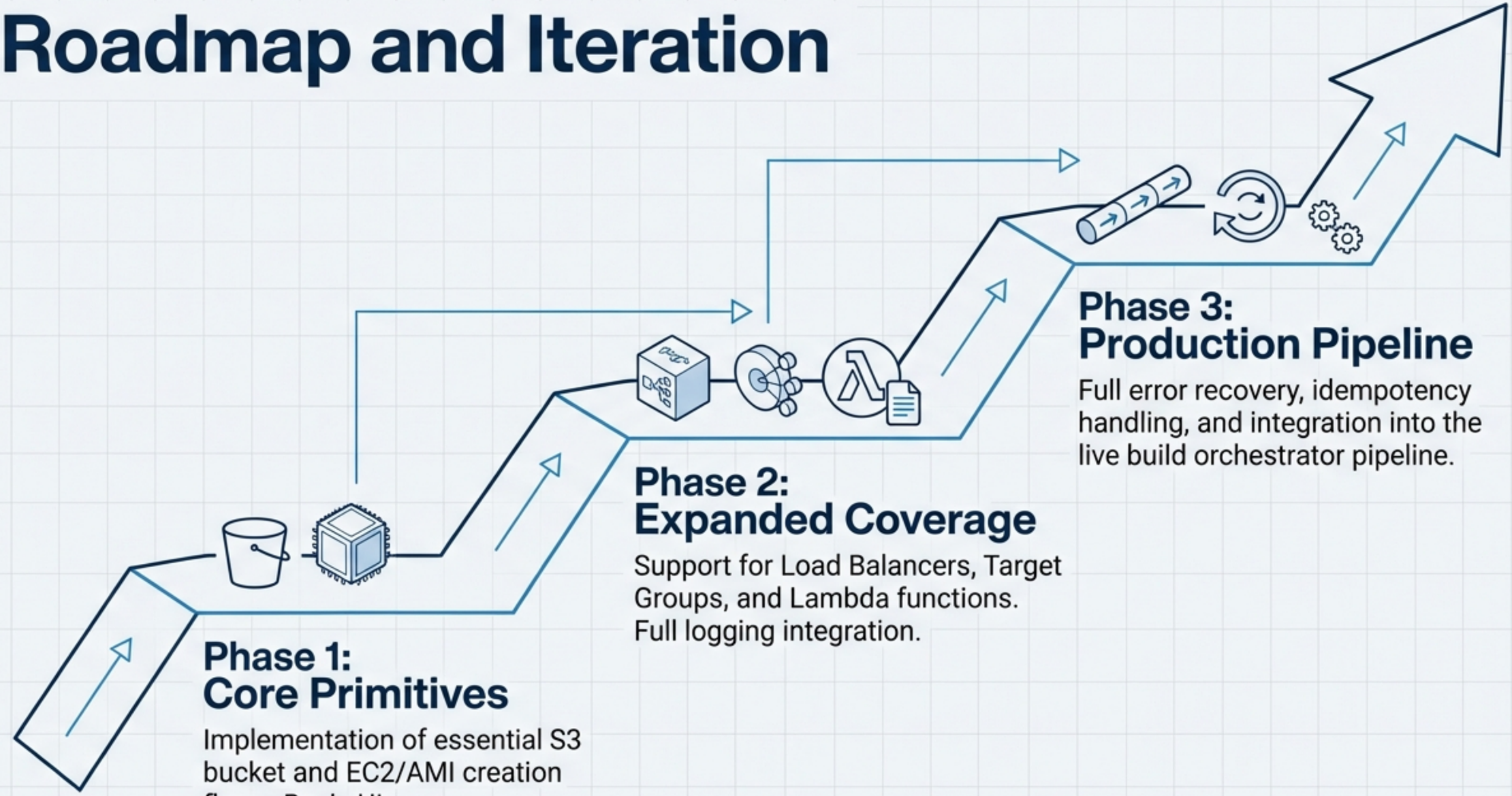
Validation of public/private key encryption flow and ephemeral credential handling.

Integration Test Suite

Automated end-to-end flows (Deploy -> Verify -> Tear Down) validated in a sandbox AWS account.



Roadmap and Iteration



Phase 1: Core Primitives

Implementation of essential S3 bucket and EC2/AMI creation flows. Basic UI.

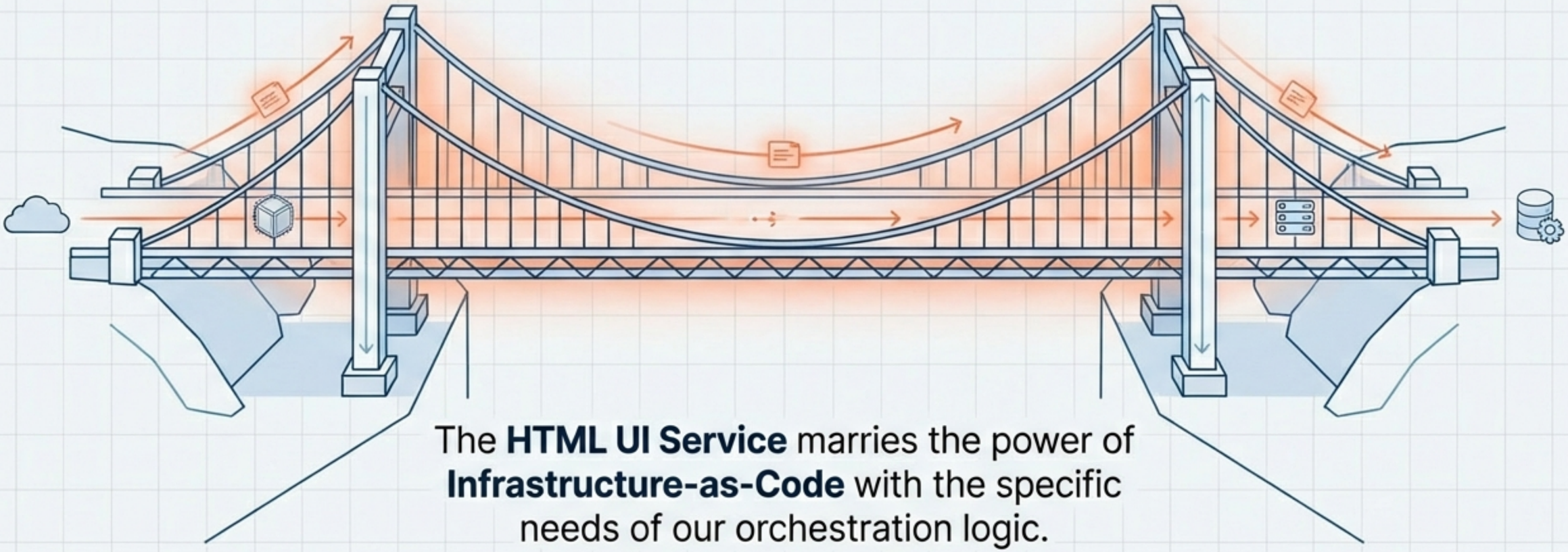
Phase 2: Expanded Coverage

Support for Load Balancers, Target Groups, and Lambda functions. Full logging integration.

Phase 3: Production Pipeline

Full error recovery, idempotency handling, and integration into the live build orchestrator pipeline.

The Missing Link in Our Infrastructure



The **HTML UI Service** marries the power of **Infrastructure-as-Code** with the specific needs of our orchestration logic.

Decoupled Logic

The ‘Brain’ guides the ‘Hands’.

Secure Execution

Ephemeral credentials, zero storage.

On-Demand Scale

Zero manual overhead for full stack provisioning.